

Bases de datos vectoriales

MMIA 6013 — IA Generativa y Agentes · Sesión 07 · martes 22 de septiembre de 2026

Semana 2 · RA-2 · de los 110 minutos, 60 son actividad y 50 son este deck

Daniel Andrés Riofrío Almeida

Ayer el vector; hoy dónde vive y cómo se encuentra

- Cerramos el lunes así: SBERT pide estructuras de índice optimizadas y no dice cuáles
- Hoy, cuáles: IVF y HNSW, las dos maneras de no mirar todos los vectores
- Una sola idea nueva, con dos encarnaciones: no hace falta mirarlos todos
- Y renunciar a mirarlos todos tiene un precio medible en tres monedas
- Recall del índice, latencia por consulta y memoria. Todo el día es esa negociación
- Hoy NO hay fragmentación (miércoles) ni métricas de recuperación (jueves)

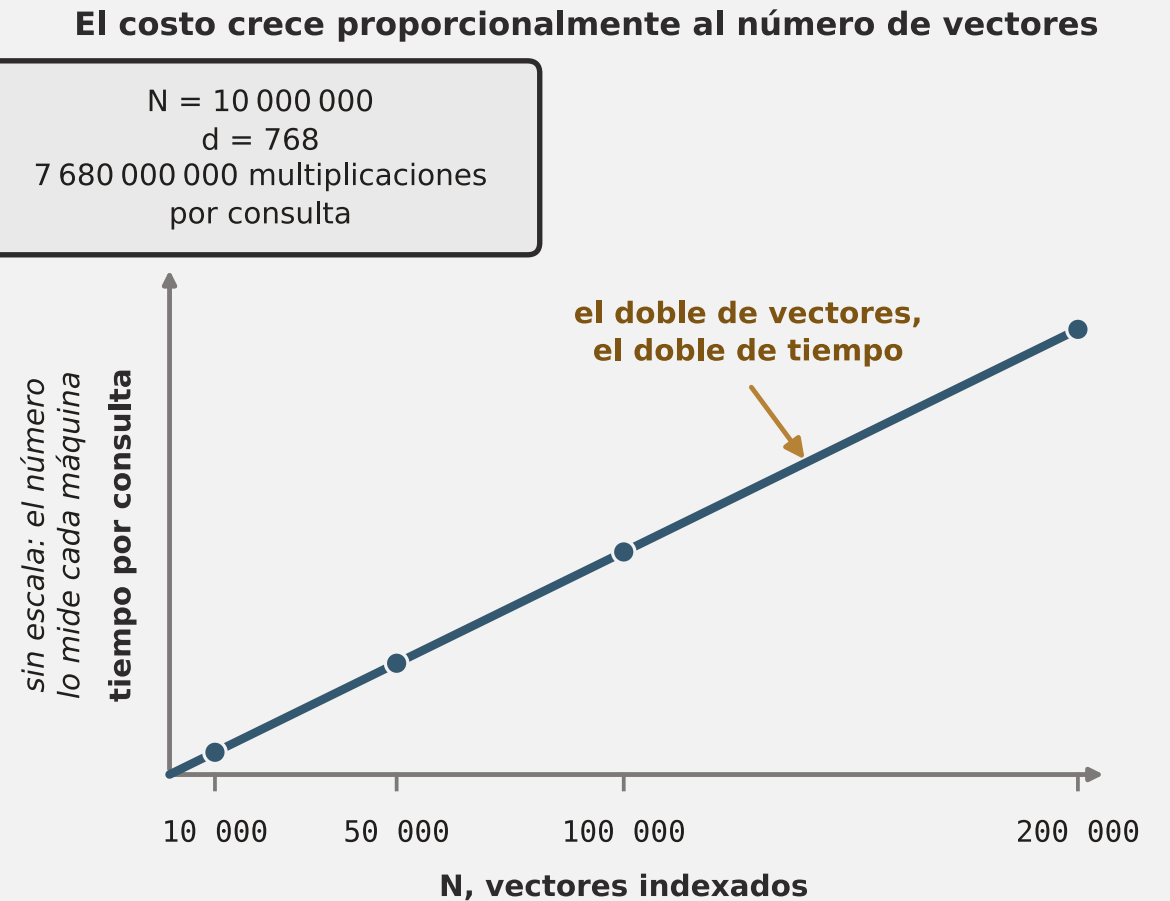
Fuente: [slides_v2/sesion-06/sesion-06.qmd](#), d21 «Lo que se fija hoy se cobra el martes, el miércoles y el sábado», notas del orador (la frase literal con la que cerró el lunes: «Sentence-BERT dice que hacen falta estructuras de índice optimizadas y no dice cuáles. Mañana, cuáles»); [curso/actividades/semana-2/s2-mar-explicacion.html](#), líneas 29–33 (objetivos de la sesión) y líneas 80–91 y 108–117 (los dos bloques de actividad con su duración explícita: 10 y 50 minutos); [curso/planificacion/plan.md](#), fila 07 (objetivo RA-2 literal) y filas 08 a 10 (chunking el miércoles, métricas el jueves, Taller 2 el sábado); [proceso/contexto/glosario.md](#) (Chunking y RAG se introducen en 08; Recall@k, MRR y Hit Rate en 09)

Por qué no se pueden mirar todos

07

El kNN exacto no escala: $O(N \cdot d)$

- Buscar exacto es comparar la consulta contra los N vectores indexados: $O(N \cdot d)$
- N es cuántos vectores hay; d es cuán largo es cada uno. El costo depende de los dos
- Con $N = 10\,000\,000$ y $d = 768$: $\sim 7\,680\,000\,000$ multiplicaciones por consulta
- Aviso de notación: N mayúscula. La n minúscula de este curso son tokens de secuencia
- La Parte 1 del notebook mide esa curva con N de 10 000 a 200 000, $d = 128$
- Lo que se proyecta es la FORMA, una recta; los milisegundos los pone su máquina



Fuente: [curso/actividades/semana-2/s2-mar-explicacion.html](#), líneas 37–38 (literal: «comparar la consulta contra todos los N vectores: $O(N \cdot d)$ »; con $N = 10\text{M}$ y $d = 768$, ~ 7.7 mil millones de multiplicaciones; aritmética reverificada el 2026-08-30: $10\,000\,000 \times 768 = 7\,680\,000\,000$); [fuentes/notas-teoricas/s2-rag-y-vector-search.md](#), §3.1, líneas 70–74 (la misma cuenta escrita como $O(n)$: el deck usa $O(N \cdot d)$, porque el costo depende de las dos magnitudes y porque n minúscula está reservada en [proceso/contexto/glosario.md](#) para la longitud de secuencia en tokens); [curso/actividades/_fuentes/s2-mar.py](#), Parte 1, líneas 29, 62 y 64–72 ($d = 128$, los cuatro tamaños de N y las 20 repeticiones por tamaño; el archivo se renumeró el 2026-09-19) y nota docente, líneas 16–17 (literal: «las mediciones de tiempo varían por máquina: lo que importa es la FORMA»)

No hay atajo exacto — y esto NO es lo que veté el lunes

- Los árboles espaciales (KD-tree) particionan el espacio por coordenadas
- En dimensión alta casi todo punto queda «cerca de la frontera» de casi toda partición
- El árbol termina visitando casi todo el dataset: rendimiento $\approx$ fuerza bruta
- Ayer veté otro uso de la misma idea: «la coseno es más robusta que la euclídea». Sigue vetado
- Son dos preguntas distintas: elegir métrica frente a poder particionar exacto
- Si hace falta escalar, la salida es renunciar a la exactitud garantizada: eso es ANN

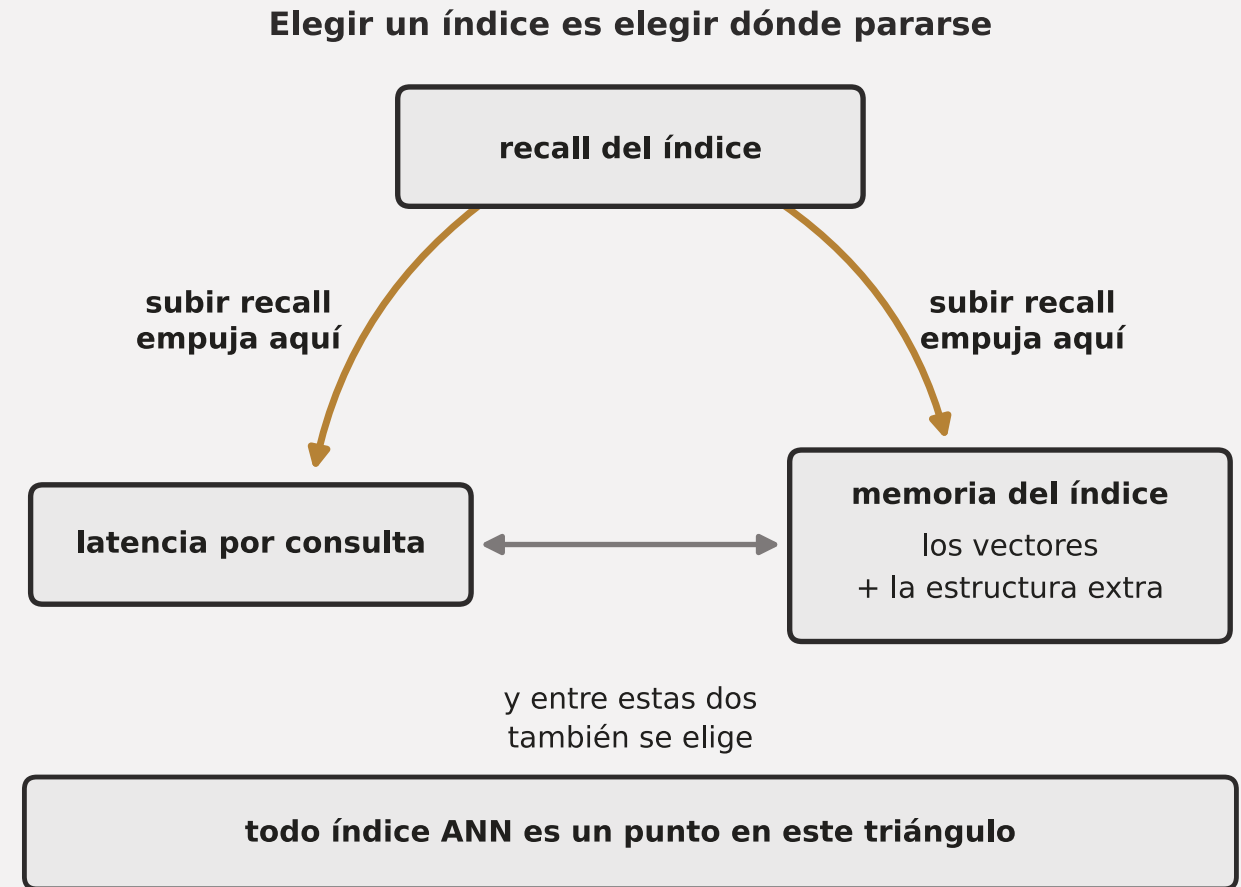
Fuente: curso/actividades/semana-2/s2-mar-explicacion.html, líneas 39–40 (literal: «en alta dimensión, los árboles clásicos (KD-tree) degeneran a fuerza bruta — la maldición de la dimensionalidad») y líneas 121–122 (autocomprobación: «casi todos los puntos quedan cerca de la frontera... el árbol termina visitando casi todo el dataset»); líneas 41–42 (definición de ANN, literal: «renunciar a la exactitud garantizada... devuelven casi siempre los vecinos correctos, en una fracción del tiempo»); slides_v2/sesion-06/sesion-06.qmd, d13 «Tres medidas, y cuándo son la misma», viñetas y notas (el argumento de la concentración de distancias NO se usa como criterio para elegir métrica), y curso/planificacion/plan.md, discrepancia 0.g, abierta. La separación explícita entre los dos papeles del mismo fenómeno es [AGREGADO - no está en las fuentes]; las dos afirmaciones por separado están citadas, y proceso/contexto/glosario.md la registra como distinción vigilada «La maldición de la dimensionalidad, en sus dos papeles»

Dos maneras de no mirarlos todos

07

El triángulo: recall, latencia y memoria

- Recall del índice: de los k vecinos verdaderos, ¿qué fracción devolvió? $0.95 \approx$ casi perfecto
- Latencia: tiempo por consulta, p50 y p99
- Memoria: la RAM del índice. Y son dos sumandos, no uno
- Los vectores mismos, más la estructura extra: el grafo o las listas
- Todo índice ANN es un punto del triángulo: más recall $\Rightarrow$ más latencia o más memoria
- Es la latencia de UNA consulta al índice, no la de la llamada completa de la semana 1

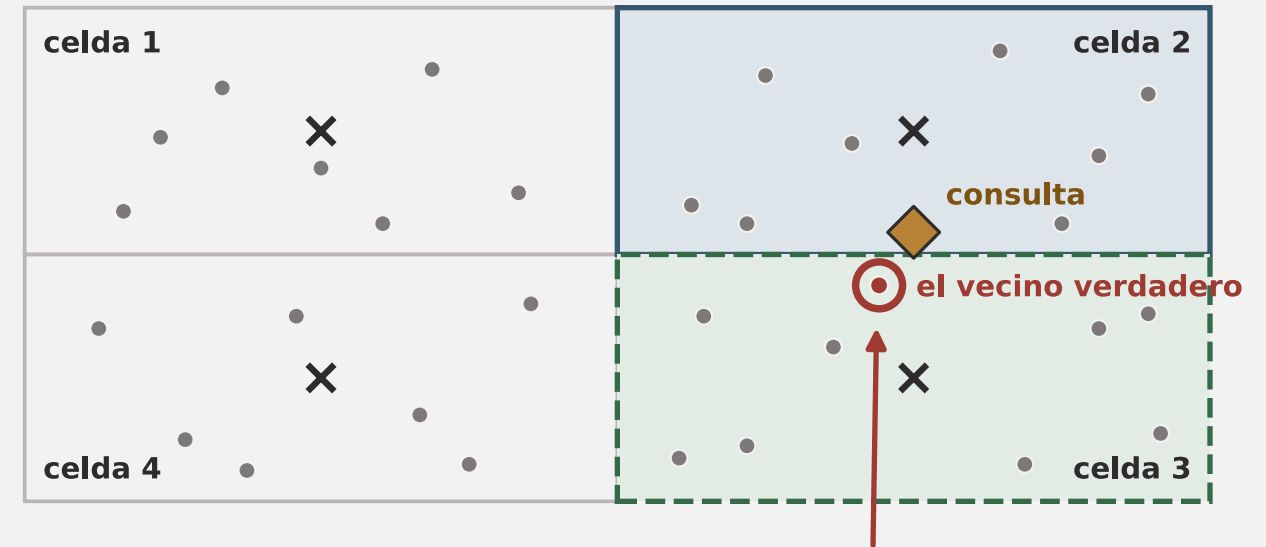


Fuente: curso/actividades/semana-2/s2-mar-explicacion.html, líneas 44–52 (las tres magnitudes son literales, incluida la definición de $\text{recall}@k$ «de los k vecinos verdaderos, qué fracción devolvió el índice», la de latencia «tiempo por consulta (p50/p99)» y la de memoria «RAM del índice (el grafo/las listas ocupan espacio extra)»; la frase de la línea 52 es literal). La separación de la memoria en dos sumandos —vectores más estructura— es [AGREGADO - no está en las fuentes]: el material dice «espacio extra» sin separarlos, y separarlos es lo que hace posible la aritmética de la diapositiva 18. [proceso/contexto/glosario.md](https://curso/contexto/glosario.md), entrada Latency (se introduce en 05: «tiempo de reloj de la llamada completa») y distinción vigilada «latencia de consulta del índice vs. latencia de extremo a extremo»

IVF: el archivador con carpetas

- Fuera de línea: k-means agrupa los N vectores en n_{list} celdas. Son las carpetas
- En la consulta, dos pasos: comparar contra los n_{list} centroides —barato—
- y buscar EXACTO solo dentro de las n_{probe} carpetas más cercanas
- La perilla y su dirección: $n_{probe} \uparrow \rightarrow recall \uparrow$, latencia $\uparrow$
- $n_{probe} = n_{list}$ ES el kNN exacto: la aproximación es un dial, no otro algoritmo
- Dónde se pierde el recall: en las fronteras entre celdas

El recall de IVF se pierde en las fronteras



con $n_{probe} = 1$ este punto no se mira

- $n_{probe} = 1$ abre esta: celda 2
- $n_{probe} = 2$ abre también ésta: celda 3
- X centroide de la celda

abrir una carpeta más recupera el vecino y examina el doble de vectores

Fuente: curso/actividades/semana-2/s2-mar-explicacion.html, §3, líneas 69–78 (la metáfora del archivador, los dos pasos de la consulta, la perilla literal « $n_{probe} \uparrow \rightarrow recall \uparrow$, latencia $\uparrow$ ($n_{probe} = n_{list} \Rightarrow$ búsqueda exacta)» de la línea 74, y la pérdida en las fronteras de la línea 76) y autocomprobación de las líneas 127–128; fuentes/notas-teoricas/s2-rag-y-vector-search.md, §3.3, líneas 87–95 (el entrenamiento de k-means es offline y la variante comprimida IVF-PQ). La notación n_{list} y n_{probe} está registrada en proceso/contexto/glosario.md; las notas escriben la fórmula con k y n , letras que el glosario reserva para otra cosa, y esa divergencia está registrada como discrepancia abierta

La aritmética de IVF, con los números del notebook

- comparaciones por consulta $\approx nlist + (N / nlist) \cdot nprobe$
- Con los valores del notebook: $N = 100\ 000$, $nlist = 64$, celdas de tamaño parejo
- $nprobe = 1 \rightarrow \sim 1\ 627$ comparaciones · $nprobe = 2 \rightarrow \sim 3\ 189$ · $nprobe = 4 \rightarrow \sim 6\ 314$
- $nprobe = 8 \rightarrow \sim 12\ 564$ · $nprobe = 16 \rightarrow \sim 25\ 064$, frente a $100\ 000$ del exacto
- Esto cuenta COMPARACIONES, no milisegundos: es una cota superior del speedup
- El código paga además concatenar índices y leer filas dispersas. El número lo miden ustedes

Fuente: fuentes/notas-teoricas/s2-rag-y-vector-search.md, §3.3, línea 92 (la fórmula del costo, reescrita aquí con nlist y N: las notas la escriben con k y n, dos letras que proceso/contexto/glosario.md reserva para top-k y para longitud de secuencia; la divergencia de notación está registrada como discrepancia abierta en curso/planificacion/plan.md); curso/actividades/_fuentes/s2-mar.py, líneas 92 y 94 ($N = 100\ 000$ y $NLIST = 64$, literales), líneas 101–108 (la implementación de knn_ivf, incluidos el np.concatenate de los índices de celda y el acceso disperso a las filas) y nota docente, líneas 16–17. La tabla de cinco filas es [AGREGADO - no está en las fuentes]: es la fórmula citada evaluada en las constantes del notebook, y toda la aritmética es verificable a mano

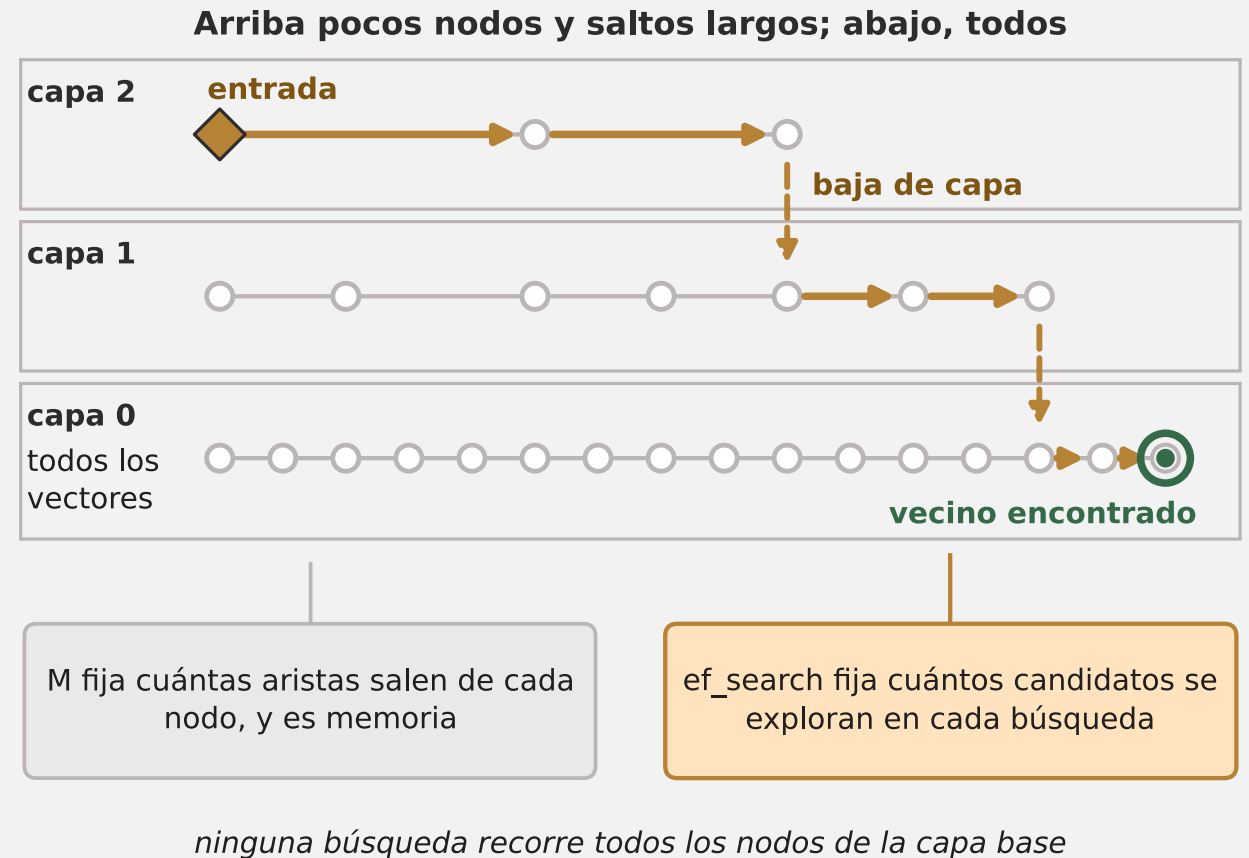
La clase como índice IVF — 10 minutos, de pie

- El aula es el índice y cada persona es un vector: su asiento es su punto
- Cuatro celdas: las cuatro esquinas del aula son los centroides
- Lanzo una consulta situada en la FRONTERA entre dos celdas
- $nprobe = 1$: levanta la mano una sola celda, la del centroide más cercano
- $nprobe = 2$: levantan la mano dos, y aparece alguien que antes no existía
- Moraleja física: el recall se pierde en las fronteras y $nprobe$ lo recupera pagando latencia

Fuente: curso/actividades/semana-2/s2-mar-explicacion.html, líneas 80–91 (actividad «La clase como índice IVF», 10 minutos, literal: el aula dividida en 4 celdas con las esquinas como centroides, la consulta lanzada en la frontera entre dos celdas y la moraleja de la línea 89). Usar la posición real de cada persona en el aula en lugar de repartir puntos 2D impresos es [AGREGADO - no está en las fuentes]: el material dice que cada estudiante «recibe (o elige) un punto 2D» (línea 83) y no incluye los puntos

HNSW: la red de aeropuertos

- Grafo de proximidad jerárquico: la capa alta tiene pocos nodos y saltos largos
- La capa base tiene todos los vectores, con conexiones cortas
- La búsqueda entra por arriba y baja de capa en capa: vuelo internacional, doméstico, taxi
- Perillas de construcción: M , conexiones por nodo, y $ef_construction$
- Perilla de consulta: ef_search , los candidatos explorados por búsqueda
- El grafo vive en RAM, construirlo es lento, insertar es barato y borrar es incómodo



Fuente: [fuentes/notas-teoricas/s2-rag-y-vector-search.md](#), §3.2, líneas 76–85 (literal: «grafo de proximidad jerárquico donde los nodos tienen conexiones largas en capas superiores y conexiones cortas en la capa base»; las tres perillas de la línea 82 y las complejidades de la línea 81: construcción $O(N \log N)$, búsqueda $O(\log N)$, reescritas aquí con N mayúscula); [curso/actividades/semana-2/s2-mar-explicacion.html](#), §2, líneas 54–67 (la metáfora de los vuelos, el esquema de tres capas —que se REDIBUJA, no se copia— y el costo literal de la línea 66: «el grafo vive en RAM y construirlo es lento; insertar es barato, borrar es incómodo»). ⚠ Vigencia: Malkov & Yashunin (2018), el paper de HNSW, NO está en [fuentes/papers/](#) ni en [papers-anotados.md](#) (verificado el 2026-08-30). Las complejidades se citan como afirmación del material docente del curso, no como resultado demostrado, y ninguna comparación cuantitativa entre HNSW e IVF se proyecta. La letra M colisiona con el M de la sesión 01 (orden de la cadena de Markov): se dice «conexiones por nodo», nunca M a secas

IVF frente a HNSW, con la fila que nadie puede llenar

- Estructura: celdas de k-means · grafo jerárquico
- Perilla de consulta: nprobe · ef_search — la misma clase de perilla en los dos
- Perilla de memoria: nlist y compresión de vectores · conexiones por nodo (M)
- Dónde pierde recall: IVF, en las fronteras · HNSW: ninguna fuente del curso lo dice
- Memoria: IVF ocupa menos · el grafo de HNSW vive en RAM
- Lo que NO digo: cuál tiene mejor curva. Aquí nadie lo midió, y el notebook solo mide IVF

Mismo tipo de control, y una casilla sin fuente		
	IVF	HNSW
estructura	celdas de k-means	grafo jerárquico
perilla de consulta	nprobe	ef_search
la misma clase de perilla en los dos		
perilla de memoria	nlist y compresión de vectores	conexiones por nodo
dónde pierde recall	en las fronteras entre celdas	ninguna fuente del curso lo dice
memoria	menos que HNSW	el grafo vive en RAM
construcción	k-means fuera de línea	lenta, y borrar es incómodo

la casilla vacía es contenido: el curso contrasta los dos mecanismos sin poder rankear su calidad

Fuente: fuentes/notas-teoricas/s2-rag-y-vector-search.md, §3.2 (líneas 76–85) y §3.3 (líneas 87–95); curso/actividades/semana-2/s2-mar-explicacion.html, líneas 65–66 (perillas y costo de HNSW), 71–77 (perilla, pérdida en fronteras y menor memoria de IVF). ⚠ curso/actividades/_fuentes/s2-mar.py, línea 133, afirma que HNSW ofrece el mismo dial «con mejor curva»: esa segunda mitad NO se proyecta — es una afirmación comparativa sin fuente, sin fecha, sin condiciones de medición y sin corpus, la misma clase de frase que la sesión 06 rechazó. Verificado el 2026-08-30: el notebook mide IVF (líneas 110–126) y no mide HNSW en ninguna parte, y ninguna fuente del repositorio dice dónde pierde recall HNSW

Recall del índice 0.99 con $k = 10$ y latencia p99 de 800 ms con HNSW. ¿Qué perilla?

Antes de responder: los dos «99» de la frase no son lo mismo

A. Subir las conexiones por nodo (M): más aristas, el vecino se encuentra antes

B. Bajar ef_search: menos candidatos explorados en cada consulta

C. Bajar nprobe: abrir menos celdas

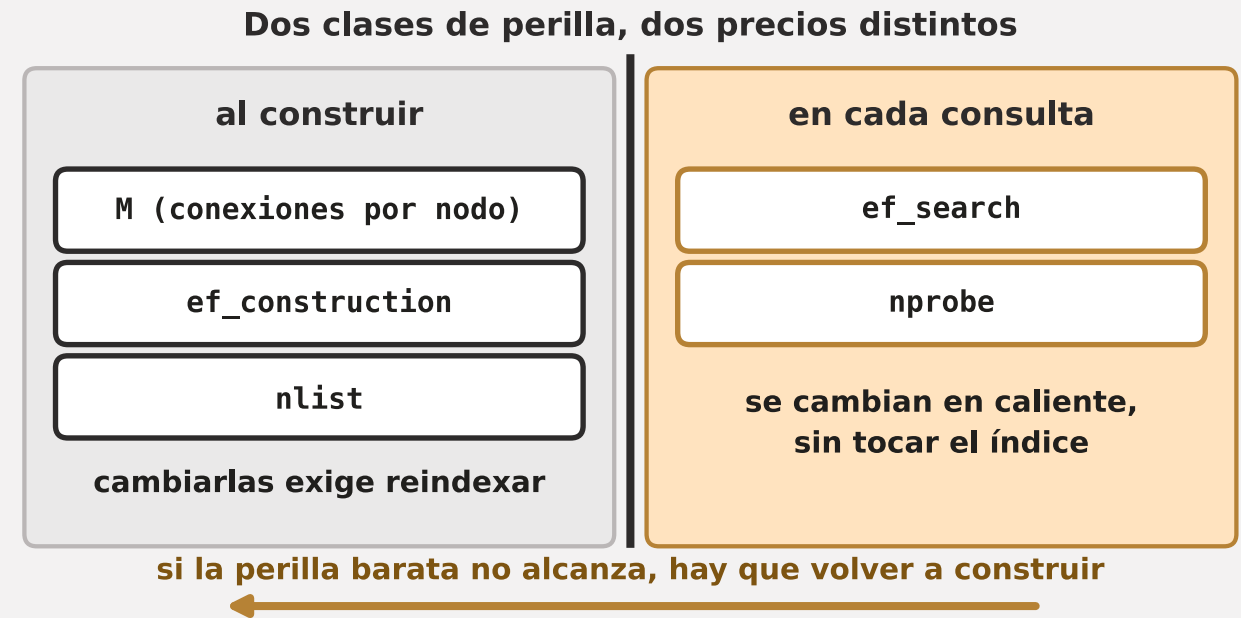
D. Reindexar con menos vectores

Y la pregunta que sigue: ¿cuál de estas cuatro se puede probar hoy mismo en producción?

Fuente: [curso/actividades/semana-2/s2-mar-explicacion.html](#), líneas 124-125 (segunda autocomprobación del material del profesor: el enunciado es literal; la respuesta, que está en la línea 125, se lee en voz alta después de recoger las manos y no se proyecta aquí). Las perillas de HNSW se presentaron en la diapositiva 11 y las de IVF en la 8. El enunciado del material escribe «recall@10»; aquí se escribe «recall del índice, $k = 10$ » porque Recall@ k está reservado en [proceso/contexto/glosario.md](#) para la métrica de recuperación de la sesión 09, que se define contra un golden set anotado. Es la misma notación por glosario que aplican la diapositiva 4 con $O(n)$ y la 9 con k y n

El compromiso se decide dos veces: al construir y en cada consulta

- De consulta: `ef_search` (HNSW) y `nprobe` (IVF). Se cambian en caliente, por búsqueda
- De construcción: `M`, `ef_construction` (HNSW) y `nlist` (IVF). Exigen reconstruir el índice
- Cambiar una de consulta cuesta nada; cambiar una de construcción cuesta reindexar
- Y las de construcción son las que fijan la memoria: dos de las tres son memoria
- El compromiso no se decide una vez: se decide al construir y otra vez en cada consulta
- Prometer más recall sin decir cuál de las dos se mueve es prometerlo gratis



🔒 aquí también se fija la memoria

solo dos de las cinco perillas se mueven sin reconstruir el índice

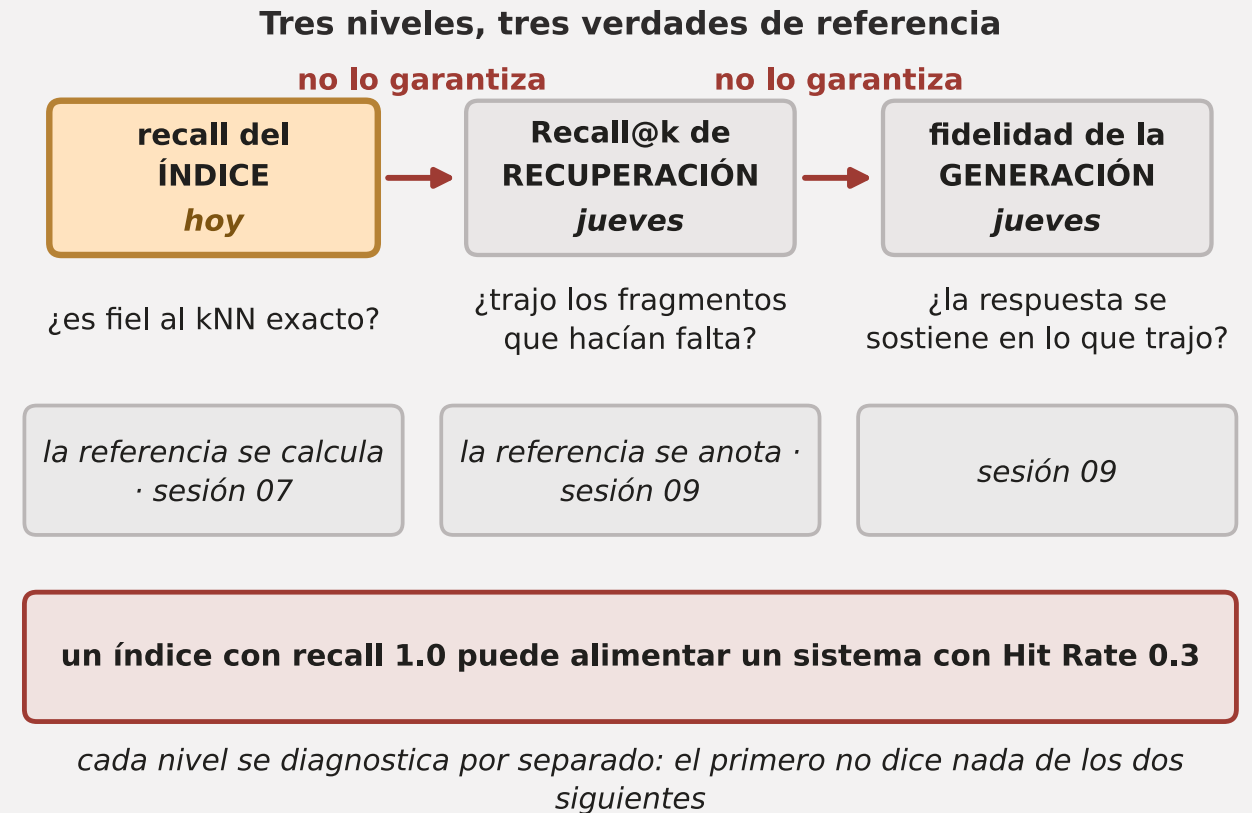
Fuente: [curso/actividades/semana-2/s2-mar-explicacion.html](#), líneas 65 (las tres perillas de HNSW y en qué momento actúa cada una), 74 (`nprobe` como perilla de consulta de IVF) y 124-125 (literal: «es ajustable en tiempo de consulta, sin reconstruir el índice... si el recall cae demasiado, subir `M` (más memoria, reconstrucción)»); [fuentes/notas-teoricas/s2-rag-y-vector-search.md](#), líneas 82 y 93. La formulación como DOS CLASES de perilla es [AGREGADO - no está en las fuentes]: las dos premisas son literales y [proceso/contexto/glosario.md](#) registra la distinción vigilada «perilla de construcción vs. perilla de consulta»

Lo que se mide y lo que se paga

07

Tres cosas se llaman «recall» y miden cosas distintas

- Recall del índice (hoy): compara lo que devolvió el índice contra el kNN exacto
- Su verdad de referencia se CALCULA: verdad = knn_exacto(consulta, base), en el notebook
- Recall@k y Hit Rate (jueves): comparan contra los fragmentos que un humano anotó
- Su verdad de referencia se ANOTA: es el golden set que pide el Taller 2
- Y encima va un tercer nivel: la fidelidad de la respuesta generada. También el jueves
- Un índice con recall 1.0 puede alimentar un sistema con Hit Rate 0.3



Fuente: curso/actividades/_fuentes/s2-mar.py, línea 116 (verdad = [set(knn_exacto(q, base)) for q in consultas]: la referencia se calcula) y línea 125 (recall = np.mean([len(a & v) / 10 for a, v in zip(aprox, verdad)]), literal: es una medida de fidelidad entre dos algoritmos); curso/labs/Lab-02-RAG-VectorSearch/evaluation.py, líneas 41–51 (es_respondible y acierta: la referencia se anota en el golden set, por documentos_fuente y fragmento_esperado) y líneas 102–103 (Hit Rate y MRR solo sobre las respondibles; versión del 2026-09-19); curso/talleres/taller-02-rag-vector-search.md, §«Parte 2.a» (el golden set, anotado por documento y frase literal) y §«Parte 2.b» (Hit Rate y MRR sobre respondibles, más las dos tasas de abstención: 10 % dentro de una Parte 2 de 30 %; revisión del 2026-09-19). Recall@k, MRR y Hit Rate se DEFINEN en la sesión 09 (curso/planificacion/plan.md, fila 09; proceso/contexto/glosario.md): hoy solo se nombran para distinguirlas. La cadena de tres niveles y el ejemplo del par 1.0 / 0.3 son [AGREGADO - no está en las fuentes]; cada uno de los tres niveles está citado por separado y proceso/contexto/glosario.md registra la distinción vigilada «Recall del índice vs. Recall@k de recuperación»

Recall del índice 0.98 y Hit Rate@5 de 0.55. ¿Subir ef_search arregla el Hit Rate?

Recall del índice 0.98 significa: casi lo mismo que mirar los N vectores uno por uno

A. Sí: 0.98 no es 1.00, todavía queda recall que ganar

B. No, o casi nada: el índice ya devuelve el 98 % de lo que devolvería el exacto, y el problema está en otra parte

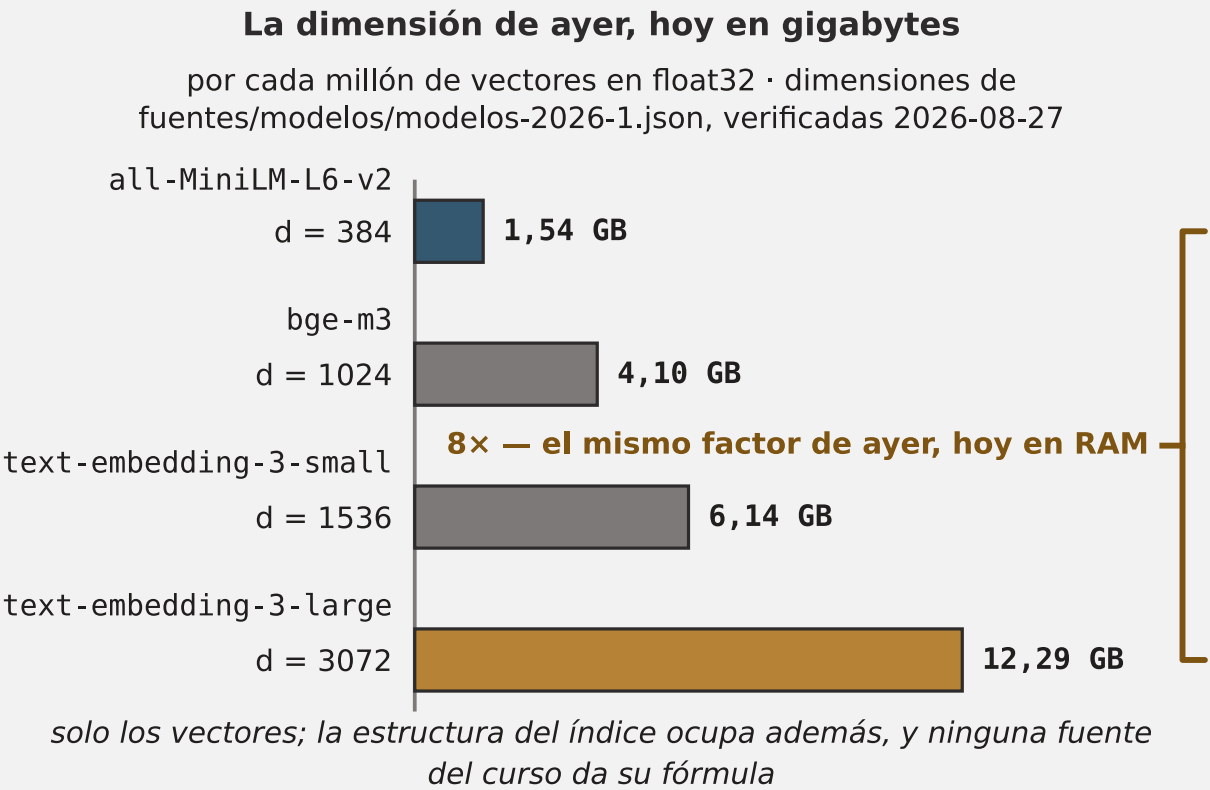
C. Antes de decidir, hay que saber si el problema está antes del índice o después

Segunda pregunta, la que vale: ¿cómo lo comprobarían, en vez de suponerlo?

Fuente: curso/actividades/_fuentes/s2-mar.py, líneas 116 y 125 (el recall del índice se calcula contra el kNN exacto); curso/labs/Lab-02-RAG-VectorSearch/evaluation.py, líneas 41-51 y 102 (el Hit Rate se calcula contra documentos_fuente y fragmento_esperado anotados, y solo sobre las respondibles; versión del 2026-09-19); curso/talleres/taller-02-rag-vector-search.md, §«Parte 2.b» (la Parte 2 evalúa lo segundo y no lo primero; revisión del 2026-09-19); proceso/contexto/rubrica_calidad.md, error grave «Reportar métricas de recuperación como si midieran la calidad de la respuesta generada»; proceso/contexto/glosario.md, distinción vigilada «Recall del índice vs. Recall@k de recuperación». El escenario numérico es [AGREGADO - no está en las fuentes]: construido para este deck sobre las dos definiciones citadas

La dimensión que eligieron ayer, en gigabytes

- Un vector es float32: ocupa $d \times 4$ bytes. El notebook lo fija con `.astype(np.float32)`
- Por cada millón de vectores, contando solo los vectores: 384 dimensiones $\rightarrow$ 1,54 GB
- 1024 $\rightarrow$ 4,10 GB · 1536 $\rightarrow$ 6,14 GB · 3072 $\rightarrow$ 12,29 GB
- Es el mismo factor 8× de ayer, ahora medido en RAM
- Y encima va la estructura del índice: ninguna fuente del curso da su fórmula
- Lo que sí consta: HNSW ocupa más que IVF, y su grafo vive en RAM



Fuente: fuentes/modelos/modelos-2026-1.json, clave embeddings, cinco filas verificadas el 2026-08-27 (all-MiniLM-L6-v2 y paraphrase-multilingual-MiniLM-L12-v2 con dimensions 384; bge-m3 con 1024; text-embedding-3-small con 1536; text-embedding-3-large con 3072). Ningún valor se escribe de memoria; curso/actividades/_fuentes/s2-mar.py, líneas 40, 96 y 155 (`.astype(np.float32)`: los vectores son float32 y eso no se supone); curso/actividades/semana-2/s2-mar-explicacion.html, líneas 49 (el grafo y las listas ocupan espacio extra), 65 (M es la perilla de memoria) y 77 (IVF ocupa menos memoria que HNSW); proceso/contexto/glosario.md, entrada Embedding dimension. La aritmética por millón de vectores es [AGREGADO - no está en las fuentes]: $d \times 4 \text{ bytes} \times 10^6$, con $1 \text{ GB} = 10^9 \text{ bytes}$, verificable a mano

Cinco preguntas antes de elegir motor

- 1. ¿Quién opera el servicio? El propio material lo pone por delante de los benchmarks
- 2. ¿Ya tiene PostgreSQL? pgvector es una extensión, no otro servicio; busca exacto por defecto y ofrece HNSW e IVFFlat
- 3. ¿Hay que filtrar, añadir y borrar en caliente? Eso es lo que un índice suelto no da
- 4. ¿Cuántos vectores? Decenas de miles: NumPy, sin índice. Millones: biblioteca ANN. El notebook midió en medio, de 10 000 a 200 000
- 5. ¿Qué usa el curso? Qdrant, en el Lab 02; Weaviate es de la misma clase. Los dos traen HNSW por defecto (docs consultadas 2026-09-19)
- Y una de la semana 1: si el requirements.txt no fija versiones, no hay línea base reproducible

Fuente: Alammar & Grootendorst (2024), cap. 8, §«Nearest neighbor search versus vector databases», pp. 238–239 — los tres escalones: NumPy sin índice hasta decenas de miles (p. 238), bibliotecas ANN a partir de millones (p. 239) y las bases de datos vectoriales como OTRA CLASE de sistema (p. 239), con Weaviate y Pinecone como ejemplos de la clase que añade y borra sin reconstruir el índice y filtra más allá de la distancia: el libro atribuye esas capacidades a la clase y no documenta ningún producto. ⚠ Son órdenes de magnitud de 2024 y se proyectan con su año, no como umbrales; el tramo de 10 000 a 200 000 no lo dicta el libro y es el que mide la Parte 1 del notebook (curso/actividades/_fuentes/s2-mar.py, línea 62). Documentación oficial consultada el 2026-09-19, registrada en fuentes/web/fuentes-web.md: Qdrant, Indexing (<https://qdrant.tech/documentation/concepts/indexing>), «Qdrant currently only uses HNSW as a dense vector index»; Weaviate, Vector Indexing (<https://docs.weaviate.io/weaviate/concepts/vector-index>), HNSW «is the default index type in Weaviate», entre cuatro tipos; pgvector, README v0.8.6 (<https://github.com/pgvector/pgvector>), «By default, pgvector performs exact nearest neighbor search», con índices HNSW e IVFFlat. Con eso cierran 0.m y la mitad (i) de 0.l en curso/planificacion/plan.md. Para las otras preguntas: curso/actividades/semana-2/s2-mar-explicacion.html, §4, líneas 93–100 (los tres motores), 102–106 (callout: el motor se elige por operación antes que por benchmarks) y 130–131 («un servicio menos que operar vale más que milisegundos de benchmark»); fuentes/notas-teoricas/s2-rag-y-vector-search.md, §3.4, líneas 97–104 (pgvector como extensión de PostgreSQL); curso/actividades/_fuentes/s2-mar.py, líneas 167–172 (el filtro es parte de la consulta) y 181–187 (la reflexión final: decidir si hace falta ANN); curso/talleres/taller-02-rag-vector-search.md, §«Parte 1», punto 3 (indexar en Qdrant, en contenedor); curso/labs/Lab-02-RAG-VectorSearch/requirements.txt (10 líneas; desde el 2026-09-19 fija mínimos con >= y ninguno con ==: la versión exacta la fija cada grupo en su entrega) y proceso/contexto/glosario.md, «línea base reproducible». ⚠ Sigue SIN proyectarse, a propósito: ningún umbral de volumen para pgvector —el README no da ninguno, y el «< 1M» de las notas y el «~1–5M» del material no tienen fuente (0.l, mitad ii, queda como «sin umbral»)– ni ningún juicio comparativo de calidad entre motores

AI notebook: 50 minutos

07

Una base vectorial es cuatro cosas, y solo una es el índice

- La colección declara dimensión y métrica: Distance.COSINE es la decisión de ayer
- Un punto es vector + payload; sin payload solo se recuperan identificadores
- El filtro no reemplaza la búsqueda vectorial: la acota
- Y lo que las sostiene a las cuatro: añadir y borrar vectores sin reconstruir el índice

```
cliente = QdrantClient(":memory:")          # modo embebido
cliente.create_collection(
    "faqs",
    vectors_config=VectorParams(
        size=64, distance=Distance.COSINE))
cliente.upsert("faqs", points=[...]) # vector + payload
hits = cliente.query_points(
    "faqs", query=embed_demo("devolucion del dinero"),
    query_filter=Filter(must=[FieldCondition(
        key="tema", match=MatchValue(value="pagos"))]),
    limit=2).points
```

Fuente: `curso/actividades/_fuentes/s2-mar.py`, Parte 3, líneas 158–172 (fuente canónica de los dos notebooks de la sesión; fragmento recortado, sin cambiar ningún identificador; la consulta de la línea 169 se abrevió por ancho de caja) y línea 201 («Qdrant: colección → upsert(vector+payload) → query con filtros»); `curso/labs/Lab-02-RAG-VectorSearch/rag_pipeline.py`, líneas 103–121 (la misma colección declarada con `size=len(vectors[0])` y puntos con `chunk_id`, `text`, `document` y `chunk_index`). Alammarr & Grootendorst (2024), cap. 8, §«Nearest neighbor search versus vector databases», p. 239 (literal: «A vector database allows you to add or delete vectors without having to rebuild the index»): es la única fuente del repositorio que enuncia esta propiedad — `curso/actividades/semana-2/s2-mar-explicacion.html`, línea 66, solo dice lo contrario en negativo y sobre el índice («borrar es incómodo»). ⚠ Vigencia: hasta el 2026-09-18 el Lab 02 usaba OTRA API para lo mismo (`recreate_collection` y `client.search`); desde el 2026-09-19 `rag_pipeline.py` habla la de este fragmento — `create_collection` en la línea 103, `query_points` en la 126— y `requirements.txt` fija `qdrant-client>=1.12` (línea 3). `create_collection` es la creación que documenta Qdrant, Collections (<https://qdrant.tech/documentation/concepts/collections>, consultado 2026-09-19, registrado en `fuentes/web/fuentes-web.md`; esa página no menciona `recreate_collection`). Discrepancia 0.k, mitad de API, cerrada en `curso/planificacion/plan.md`. Credenciales: verificado el 2026-09-19, la actividad corre entera en local y no usa ninguna clave; en el Lab 02, `.env.example` lleva las claves vacías y `rag_pipeline.py` lee todo con `os.getenv` tras `load_dotenv` (`curso/politica-credenciales.md`)

La Parte 3 usó una base vectorial de verdad. ¿Qué vértice del triángulo midió?

- A. Recall: recuperó la FAQ correcta con una consulta escrita en otras palabras
- B. Latencia: la consulta volvió en milisegundos
- C. Memoria: 64 dimensiones por punto, y se puede calcular
- D. Ninguno de los tres

Pista: ¿de dónde salieron los vectores de esa celda?

Embebido el martes, contenedor el sábado

- Hoy: QdrantClient(":memory:") — sin infraestructura y sin persistencia; muere con el kernel
- El sábado: QdrantClient(url=...) contra un servicio en localhost:6333. El enunciado lo exige; :memory: es solo para depurar
- Ese servicio se levanta con docker run -p 6333:6333 qdrant/qdrant
- Cambia una línea de código y aparece una pieza de infraestructura que hay que operar
- Ninguna sesión de esta semana practica Docker, y el sábado el contenedor es requisito: por eso es tarea de hoy
- Tarea: Docker instalado, la imagen qdrant/qdrant descargada y el contenedor arrancado una vez ANTES del sábado

Una línea de código y aparece infraestructura

	hoy, el notebook	el sábado, el Taller 2
cliente	QdrantClient(":memory:")	QdrantClient(url=...) contra localhost:6333
infraestructura	ninguna	docker run -p 6333:6333 qdrant/qdrant
persistencia	ninguna, muere con el proceso	la del contenedor
qué puede fallar	nada, ya funcionó	el puerto, la imagen sin descargar, el demonio apagado

ninguna sesión de la semana practica esto, y el sábado vale 25 %

Docker instalado y la imagen descargada ANTES del sábado

Fuente: curso/actividades/_fuentes/s2-mar.py, línea 158 (QdrantClient(":memory:")) y nota docente, líneas 20–21 (literal: «Qdrant en :memory: no necesita Docker — ideal para clase; el server Docker queda para el taller del sábado»); curso/labs/Lab-02-RAG-VectorSearch/README.md, líneas 17–20 (docker run -p 6333:6333 qdrant/qdrant, y «:memory: es la misma API sin contenedor: sirve para la Parte 0 y para depurar») y rag_pipeline.py, líneas 74 y 87 (QDRANT_URL con http://localhost:6333 por defecto, y :memory: si se pide); curso/talleres/taller-02-rag-vector-search.md, §«Parte 0» (corre «sin Docker», con QDRANT_URL=":memory:") y §«Parte 1», punto 3 (literal: «Indexa en Qdrant (el contenedor, no :memory:, que es solo para depurar)»). Desde la revisión del 2026-09-19 la tarea ya no es recomendación de este deck: el contenedor es requisito declarado del enunciado, y la discrepancia 0.k está cerrada en curso/planificacion/plan.md. Lo que sigue siendo cierto: ninguna sesión de la semana practica Docker, y el Taller 2 vale 25 %

El sábado se evalúa el sistema, no el índice

- Taller 2 — sistema RAG sobre base vectorial, 25 %, sábado 26 de septiembre
- El taller pide indexar en Qdrant; no pide elegir motor ni mover perillas
- Sirve sobre todo para lo contrario: con decenas de miles de vectores, NumPy es la respuesta
- Tarea: Docker y la imagen listos; qdrant-client instalado; grupo y corpus decididos
- Mañana: fragmentar y recuperar — y ahí está el cuello de botella de verdad
- La base vectorial rara vez es lo que limita la calidad. Hoy vimos por qué, y qué sí limita